

ORIGAMI AS A SPATIAL AND GEOMETRIC REASONING BENCHMARK

Anonymous authors

Paper under double-blind review

ABSTRACT

Multimodal language models perform well on visual recognition and on tool use, but these abilities do not transfer directly to spatial and geometric reasoning. Geometric structure does not reduce to text or to a single image: exact angles and incidences, which surface lies in front of which, and the order in which operations were applied are not recoverable from a semantic description of a scene. We introduce **CP2Seq**, a benchmark for evaluating geometric reasoning and sequential planning in multimodal language models through origami. The benchmark contains 600 procedurally generated samples, each requiring a model to produce a valid sequence of folds from a crease pattern and a target folded state. Samples are produced by a deterministic folding engine. Ground truth is recorded as each fold happens rather than annotated afterwards. We release the generation and evaluation framework to support additional samples and experiments across reference depths and action models. During evaluation, the engine executes proposed folds or rejects invalid actions with explicit feedback, while the model remains responsible for selecting actions and searching for a solution. We evaluate multimodal language models under progressively greater tool assistance, and against a deterministic breadth-first search over the same action set. Candidate sequences are assessed by executing them and comparing the resulting folded state with the target, allowing for planar translations, rotations, and reflections. Under the low reasoning effort setting, none of the evaluated models solves a sample outside the easy subset. On that subset, the best model achieves a success rate of 20.4%, compared with 54.7% for the search baseline. Its dominant failure mode is not an exhausted query budget but the repeated proposal of states it has already visited, which terminates 48% of its attempts. These results identify a substantial gap between multimodal language models and explicit search on this origami planning task.

1 INTRODUCTION

Geometric planning requires a model to connect a description of a target with the operations needed to construct it. In origami, this involves choosing a fold line, identifying which parts of the sheet should move, and tracking how the resulting reflections change their positions and layer order. A locally valid fold may lead away from the target, so completing a sequence also requires selecting among alternatives and revising earlier decisions. We study whether multimodal language models can use visual and geometric observations to make these decisions over successive folds.

Origami is an unusually economical instrument for this. A single sheet and a small action space generate geometric dependencies of arbitrary depth at no annotation cost, so difficulty is set by fold depth rather than by collecting harder images, and the exact simulator makes every intermediate state checkable rather than merely plausible. That combination of cheap depth and exact step-level checking is what a benchmark needs if it is to support not only measurement but the iteration that improves spatial and geometric ability: an automatic, dense signal over a long horizon, which visual question answering over static scenes does not provide.

Existing origami benchmarks assess some aspects of geometric understanding, but answering questions about a folded configuration does not establish the ability to construct it. GamiBench (Spencer et al., 2025) uses visual question answering to assess folded configurations, viewpoint consistency,

and impossible patterns. OrigamiSpace (Xu et al., 2025) includes spatial reasoning tasks and crease-pattern code generation with an interactive environment. Generating a pattern specification differs from constructing a sequence of folds that realizes a supplied target. CP2Seq evaluates whether a model can translate geometric information into a complete, executable folding sequence, with each action checked for validity and the resulting construction checked against the target. Given a crease pattern and target folded state, the model must select folds, track their effects, and revise its choices until execution reproduces both the crease pattern and the target state.

We introduce **CP2Seq**, a benchmark of 600 procedurally generated origami samples and a framework for generating, executing, and evaluating folding sequences. Each task supplies a crease pattern and a target folded state; the reference sequence and intermediate reference states are withheld. Starting from an unfolded square, the model must produce a valid sequence of simple folds under the specified action model. In the visual setting, it receives images of both inputs and views of its current folded state. The crease pattern constrains which creases must be produced, while the target specifies the arrangement of the folded sheet, including layer order. Together, these inputs guide the model’s choice of where to fold and which layers to move.

The task combines *geometric reasoning*, *spatial state tracking*, and *sequential planning*. A fold line is specified in the current folded coordinates, whereas the target crease pattern describes the original sheet. Connecting these representations requires tracking reflections and the correspondence between paper regions. Selecting a fold also requires reasoning about overlapping layers and their connectivity. Subsequent decisions depend on the state produced by earlier actions, so increasing sequence depth extends the planning horizon. We evaluate these abilities through the construction of valid folding sequences across increasing reference depths. The environment represents flat-folded states with explicit layer ordering.

We generate examples procedurally by applying legal folds and recording the resulting sequence, crease pattern, and folded state. This supplies reference solutions without manual annotation and lets us vary reference depth and the permitted folding operations. Releasing the generator also enables fresh evaluation instances beyond the published corpus, reducing reliance on fixed examples that future models may encounter during training (Section 3).

We also provide an evaluation framework that presents images of the input crease pattern, the target folded state, and the current state after each accepted edit. These observations let the model compare its progress with the target and choose its next action. Tools support adding and removing folds, inspecting geometry, and returning to a specified earlier step to explore another sequence. The engine checks proposed edits and returns feedback, while the model directs the search. Different levels of tool assistance allow us to study how this support affects performance (Section 4.3).

Evaluation checks both individual actions and the completed construction. Candidate sequences are replayed and assessed for crease-pattern agreement and agreement with the target folded state. A sequence need not match the recorded actions to be correct. Folded-state comparison accounts for a global translation, rotation, or turnover of the completed object, with the corresponding changes to layer order and face orientation (Section 4.2). We also provide a deterministic breadth-first search baseline using the same candidate-fold enumerator and terminal-state comparator. This baseline measures how much can be achieved by systematic search with the available filtering, providing context for model performance beyond comparisons between language models.

Our contributions are:

1. **A dataset and generation framework.** CP2Seq contains 600 samples with crease patterns, target states, recorded fold sequences, and intermediate states. Configurable generation supports experiments across reference depths and action models, with coupling recorded to characterize geometric structure.
2. **An interactive evaluation and verification pipeline.** The framework connects visual and geometric inputs to tools for constructing and revising sequences, step-level legality checks, and final replay-based scoring. It includes corpus checks and a folded-state comparator. Experiments vary access to legal-fold enumeration and target-comparison feedback.
3. **A search baseline and an empirical analysis of model behavior.** We provide breadth-first search over the same candidate actions, report performance by difficulty group, and

analyze episode termination. A broader evaluation of `gpt-5.6-luna` is complemented by smaller ablations of model version, reasoning effort, and tool assistance.

Our experiments comprise a broader evaluation of `gpt-5.6-luna` and a smaller ablation study targeting ten samples from each of the easy, medium, hard, and some-layers groups per configuration. The ablations compare `gpt-5.6-luna` at low reasoning effort with `gpt-6-luna` at low and high effort, using basic editing and observation tools, legal-fold enumeration, or enumeration with automatic target feedback. In this ablation set, high-effort `gpt-6-luna` solves easy and some-layers examples, but no completed Luna attempt solves a medium or hard example. Breadth-first search provides a separate baseline over the enumerated action space. Together, these evaluations examine sequence recovery across model settings and levels of tool assistance.

2 RELATED WORK

2.1 ORIGAMI BENCHMARKS AND GENERATION

GamiBench (Spencer et al., 2025) evaluates multimodal models through visual questions about folded configurations, viewpoints, and impossible crease patterns. Its viewpoint-consistency and impossible-fold-selection metrics assess whether predictions remain consistent across views and whether models recognize infeasible patterns. OrigamiSpace (Xu et al., 2025) evaluates pattern prediction, multi-step spatial reasoning, spatial relationships, and crease-pattern code generation. It also provides an interactive environment for the code-generation task. These benchmarks connect origami geometry to visual reasoning and generation, with different requirements on the model’s output.

CP2Seq focuses on recovering a complete folding sequence from a crease pattern and a target folded state. The model selects and revises actions, and evaluation checks their execution and the resulting construction. This differs from selecting a folded configuration or generating a pattern specification. Our procedural generator records reference sequences as folds are performed, allowing new instances to be produced without manually annotating solutions. The environment checks proposed actions under its supported folding model and supplies feedback during the search. Rejecting an invalid action in a reachable instance is distinct from identifying an impossible crease pattern: the former concerns a particular move, while the latter concerns the feasibility of the pattern itself.

2.2 COMPUTATIONAL ORIGAMI AND SEQUENCE PLANNING

A *simple fold* is the operation introduced by Arkin et al. (2004) for map folding: the sheet is folded along one line at a time, by $\pm 180^\circ$. The models are distinguished by how much of the stack that line carries, namely a single layer, a contiguous run from the top or bottom (some-layers), or the entire stack (all-layers). A crease pattern is *simply foldable* in a given model when some sequence of such folds flattens the sheet and leaves exactly its creases. Deciding that is a studied question whose difficulty depends on the shape of the sheet, the directions the creases take, and which of the models is permitted (Arkin et al., 2004; Akitaya et al., 2017).

CP2Seq uses the some-layers and all-layers variants. Work on flat-folded states studies layer ordering as well as planar geometry (Bern & Hayes, 1996; Akitaya et al., 2026), motivating our explicit representation of layers in the state and final comparison. Earlier work also addresses generating folding sequences from crease patterns (Akitaya et al., 2013); sequence recovery is therefore an established computational origami problem. We use it to evaluate multimodal models through visual observations, executable actions, and feedback.

3 CP2SEQ: TASK AND DATASET

3.1 TASK AND ACTION MODELS

Given a crease pattern C and a target folded state S^* , the model must construct a sequence of legal folds $a_1, \dots, a_T$ starting from an unfolded square. Replaying the sequence must reproduce the crease pattern and reach the target state under the equivalence defined in Section 4.2. Each state

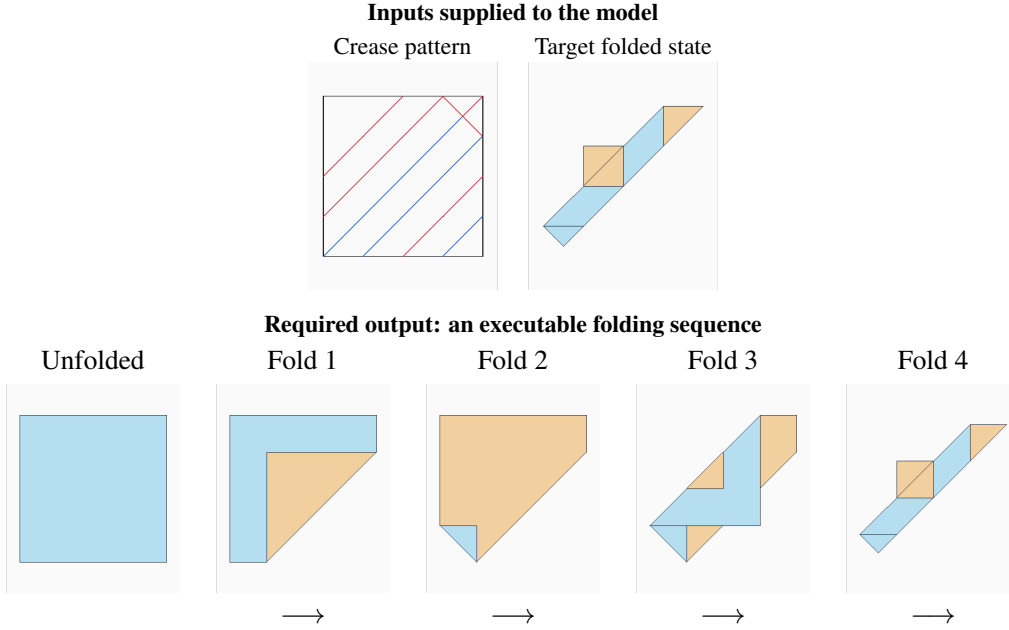


Figure 1: CP2Seq task illustrated by all-layers/easy-0001. The model receives the crease pattern and target state and must produce a valid sequence from the unfolded sheet. The lower row shows states from the recorded reference sequence for illustration, not a model prediction; this sequence and its intermediate states are withheld during evaluation.

records paper geometry and layer ordering; the reference sequence and its intermediate states are withheld, and alternative valid sequences are accepted. Figure 1 summarizes the task.

We use two simple-fold action models (Arkin et al., 2004): all-layers folds move the paper on the selected side of a fold line across the full stack, while some-layers folds select a contiguous run from the top or bottom of the stack. Both use flat 180° folds, with connectivity checks to reject moves that would tear the sheet; state representations and action constraints are detailed in Appendix A.

3.2 DATASET CONSTRUCTION

Starting from a square sheet, the generator samples a legal fold under the chosen action model, applies it, and records the action and resulting state. This process continues to the requested reference depth; attempts with no available legal continuation are discarded. The accumulated creases are mapped to the original sheet to form the input crease pattern, with edges split at intersections. The last state supplies the target, and the recorded actions supply a reference solution. We group samples by reference fold count, hypothesizing that longer sequences generally require more search and are therefore harder to recover. These groups describe expected difficulty; reference length need not equal the shortest valid solution length. We also record coupling, the number of creases created by a fold, to characterize the geometry of each instance.

The generation procedure is summarized below.

```

GENERATE(reference_depth, action_model):
    state <- unfolded square
    actions, frames <- [], []
    for step in 1..reference_depth:
        candidates <- LEGAL_FOLDS(state, action_model)
        if candidates is empty: return FAILED
        action <- SAMPLE(candidates)
        state <- APPLY(state, action)
        actions.append(action)
        frames.append(state)
    cp <- CREASE_PATTERN_IN_ORIGINAL_SHEET(state)

```

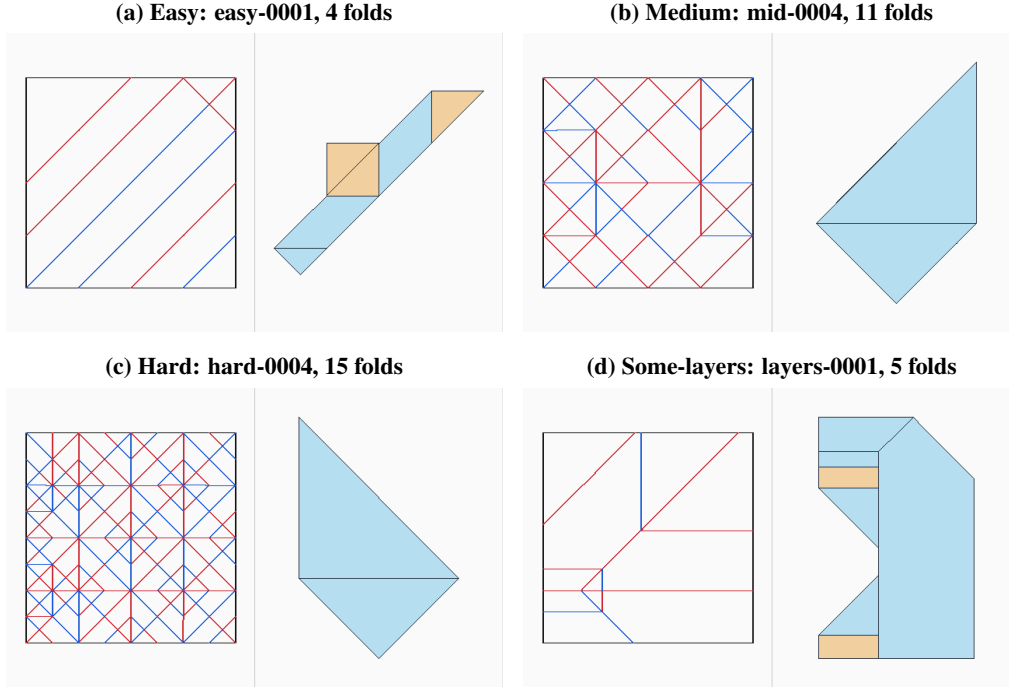


Figure 2: CP2Seq examples: each panel pairs the crease pattern (left) with the target top view (right). Panels (a–c) use all-layers folds; (d) is from some-generated-d5. Counts are reference folds. Red and blue denote mountain and valley creases.

```
cp <- SPLIT_EDGES_AT_INTERSECTIONS(cp)
if not REPLAY_CHECK(cp, actions, state): return FAILED
return (cp, state, actions, frames, METADATA(actions))
```

We provide inspection software to visualize crease patterns and folded states, replay reference sequences, and examine verification feedback. These views support manual review to identify samples with inconsistent geometry or failed checks for exclusion from the dataset. Example interfaces and rejection feedback appear in Appendix B.

3.3 DATASET STATISTICS

CP2Seq contains 600 samples: 400 all-layers examples grouped by reference depth and 200 some-layers examples with three to six reference folds (Table 1). These counts describe distinct dataset samples, not evaluation attempts.

Table 1: Dataset composition. Fold counts refer to recorded reference sequences.

Action model	Group	Samples	Reference folds
All-layers	Easy	200	4–10
All-layers	Medium	100	11–13
All-layers	Hard	100	14–19
Some-layers	Depth 3	50	3
Some-layers	Depth 4	50	4
Some-layers	Depth 5	50	5
Some-layers	Depth 6	50	6
Total		600	3–19

Each sample contains the task inputs, a reference sequence, and metadata (Table 2). The model receives the crease pattern and target state; reference actions and intermediate reference states are withheld. Tool availability and target feedback are configured separately from dataset groups (Section 4.4.1).

Table 2: Contents of a CP2Seq sample.

File	Contents
<code>cp.fold</code>	Input crease pattern.
<code>steps.fold</code>	Folded states after each reference action; only the final state is supplied as the target.
<code>seq.json</code>	Withheld reference actions: fold lines, directions, layer selections, and creases created.
<code>meta.json</code>	Sample descriptors and generator version.

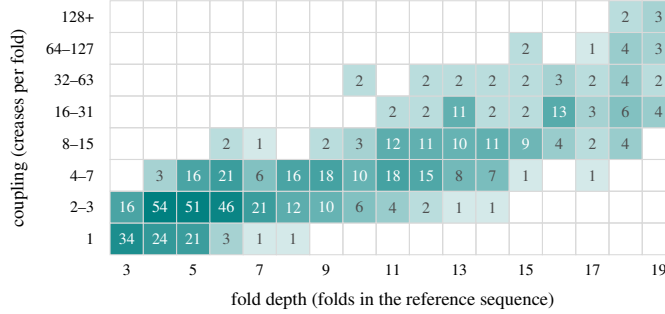


Figure 3: **What the corpus covers, over fold depth and coupling.** Each cell holds the number of samples at that depth and coupling, where coupling is creases made per fold, binned by doublings because an all-layers fold that crosses a doubled stack makes twice the creases. The two axes are not independent: a fold of all layers thickens the stack, so a long sequence cannot keep cutting few layers. No sample reaches 15 folds at coupling below 4, and the deepest such sample has 14; reaching that corner in practice takes pre-creasing, which this action space excludes. The corpus is therefore thinner than real folding in a way the difficulty groups alone do not show.

The two replay-based corpus checks pass on all 600 samples; their shared-engine validation scope is discussed in Section 6.

Coverage is limited in two respects: long reference sequences with low coupling are sparsely represented by the generation procedure, and some-layers examples extend only to six reference folds. These are limits of the present corpus, not proofs that other configurations are impossible.

4 EXPERIMENTAL SETUP

4.1 ENVIRONMENT

The model edits a folding sequence through the tools in Table 3. For each proposed fold, the engine reflects the selected paper across the fold line, updates layer order and orientation, and checks sheet connectivity and compatibility with the input crease pattern. Accepted edits update the current state; rejected edits return an error code and preserve the previous state.

For a some-layers fold, moving and stationary regions may share an edge away from the hinge. Reflecting only the moving region would separate that shared edge, so the engine rejects the proposal as `would-tear`. A `no-crease` rejection means that paper was selected to move but the operation did not create a crease segment; it is distinct from the connectivity check. Other rejections cover an empty moving region, an incompatible over/under direction, and creases absent from the target pattern. That last check is a form of assistance: it prevents a class of unrecoverable moves, and the rates reported here are therefore rates under that constraint. Detailed conditions are listed in Appendix A.7.

The environment never refuses a fold for self-intersection. A fold moves either the whole stack or a contiguous run taken from the top or the bottom, and such a move cannot drive paper through the layers it leaves behind, so the action space excludes self-intersection by construction.

Observation tools are listed in Table 3. Exploded views use illustrative layer separation rather than simulated paper thickness.

The repository also provides a separate Flat-Folder check for the existence of a flat-folded state consistent with a crease pattern (Akitaya et al., 2026). This check is distinct from action execution and target matching, takes no part in scoring, and is not invoked during an episode; its scope is described in Appendix A.7.

4.2 EVALUATION METRICS

The recorded sequence provides one solution, but other sequences may produce the required construction. We therefore assess correctness by execution rather than agreement with the reference actions. Distance between the constructed crease pattern and the input crease pattern is a separate measure of geometric progress; it does not compare action sequences or replace final-state verification.

4.2.1 VERIFICATION

We execute the proposed sequence and compare its accumulated creases with the input pattern, including mountain–valley assignments. We then compare the final stack with the target after alignment by a single global translation, rotation, or turnover. A turnover reflects the geometry, reverses the stack order, and flips face orientations. The same transformation must align all layers; independent layer transformations and arbitrary layer permutations are not accepted. A successful construction must satisfy both crease-pattern and folded-state agreement.

Neither comparison is tolerance-free. The crease comparator and the shared folded-state comparator both match positions within 2×10^{-6} ; Table 9 lists every tolerance in the pipeline, and Appendix A.6 describes the alignment and representation details. These final checks are distinct from the three optional levels of target feedback supplied during an episode.

4.2.2 REPORTED METRICS

We report solve rate by fold-count group, with the number of attempts and the applicable run budgets. Termination reasons distinguish successful submission, unsuccessful completion, budget limits, and repetition-related stopping. Query counts describe search effort; for unsuccessful attempts they are consumed queries, not observed queries to solution.

We measure CP edit distance between the accumulated crease pattern in the last recoverable state and the supplied pattern, in original-sheet coordinates. Collinear mountain and valley intervals are merged separately, excluding sheet boundaries. If m intervals match under the crease comparator, the insertion/deletion distance is $d_{CP} = |C_{target}| + |C_{candidate}| - 2m$. Each missing or extra interval costs one; changing an assignment costs two. Partially overlapping intervals do not match unless their endpoints agree within tolerance. We also report $d_{CP}/|C_{target}|$, which can exceed one. Normalized distance is undefined for an empty target. Means weight attempts equally and exclude missing state artifacts, whose counts are reported separately. This metric compares crease patterns, not fold sequences, and zero distance does not establish folded-state agreement.

The comparator that scores every reported run is validated by a test that must accept the global transformations and reject everything else. Acceptance is checked across rotations, turnovers and translations, in both comparison directions, and under changes to polygon start vertex and winding that do not change the folded object. Rejection is checked for a turnover without stack reversal, a turnover without parity flips, a coordinate mirror presented as a turnover, a permuted layer order, a missing layer, a uniform scaling, a per-layer transformation that no single isometry explains, and an empty stack. The polygons used are asymmetric, so an incorrect reflection cannot pass through a symmetry of the square. This validates the implementation on the scoring path. The corpus-side comparator is a separate implementation with its own negative control, described in Appendix A.6, and does not stand in for it.

4.3 EVALUATION PROTOCOL

The model interface uses OpenAI Codex CLI (OpenAI, 2026) through a custom episode harness. Software version evidence is documented in Appendix B.1.

4.3.1 EPISODE PROCEDURE

One episode is one sample attempted by one model under one tier of assistance. The loop is the same for every arm and every model, and the only thing that varies between arms is what the model is allowed to see and call.

```
EPISODE(sample, model, condition, query_budget):
    cp, target <- LOAD_INPUTS(sample)
    session <- INITIALIZE(cp, target, unfolded_square)
    history <- []
```

```

378   for query in 1..query_budget:
379       obs <- OBSERVE(session, condition)
380       call <- model(cp, target, obs, history, condition)
381       if call is finish: break
382       # Edits: add/remove fold, go to step, restore revision.
383       # Reads: state, images, candidates, target feedback.
384       result <- DISPATCH_TRANSACTIONALLY(session, call)
385       # Rejected edits leave session unchanged but consume a query.
386       # Read calls consume a query without changing the sequence.
387       history.append(call, result)
388       if STOPPING_RULE(session, history): break
389   return SCORE(REPLAY(session.actions), REFERENCE_AT_SCORING(sample))

```

The pseudocode describes one tool call per query. Accepted edits update the current sequence; rejected edits preserve it, and both consume budget. Removing a fold replays the remaining actions; returning to an earlier step truncates the sequence, while revision restoration can recover a previous branch. The reference sequence is withheld until scoring. Run-specific turn, time, and cycle limits must be reported alongside the query count.

The evaluation pipeline is shown in Appendix B.3, Figure 6.

4.3.2 TOOL CONFIGURATION

Table 3 lists the tool interface. Basic tools comprise all entries except the optional `list_legal_folds` and `compare_to_target` tools. Loading and configuration are described in Appendix B.7.

Table 3: Tools available to the evaluation harness; exposure depends on run configuration.

Tool	Operation
<code>add_fold</code>	Append a checked fold; reject invalid actions without changing state.
<code>remove_fold</code>	Remove a specified fold and replay the remainder transactionally.
<code>go_to_step</code>	Retain the first specified number of folds.
<code>restore_revision</code>	Restore a previously recorded branch state.
<code>get_state</code>	Return the current sequence and layer geometry.
<code>get_images</code>	Render top, oblique, reverse, X-ray, and exploded views of a completed step; exposed views depend on run configuration.
<code>list_legal_folds</code>	Optionally enumerate legal candidates compatible with the target CP.
<code>compare_to_target</code>	Optionally return feedback at the configured level (Section 4.4.1).
<code>finish</code>	End the episode and evaluate the current sequence.

4.3.3 MODELS AND BUDGETS

The recent ablations evaluate GPT-5.6 Luna at low reasoning effort and GPT-6 Luna at low and high effort, using ten samples per group and configuration. Runs use an 80-turn limit, a 300-second call timeout, and full image history. Earlier runs have different coverage and budgets and are reported separately (Section 5).

4.3.4 SEARCH BASELINE

Deterministic breadth-first search uses the same legal candidate enumerator and terminal comparator as the model interface, without access to the reference sequence. It runs under per-sample wall-clock, state-expansion, and depth limits. Section 5.3 reports measured outcomes; enumeration details and search-effort plots appear in Appendix B.2.

4.4 ASSISTANCE CONDITIONS

The recent ablations compare basic tools, legal-fold enumeration, and enumeration with automatic tier-3 target feedback. Earlier runs also include on-demand target comparison. Table 4 specifies the additions to the basic interface defined in Table 3.

A controlled comparison should hold the model, sample set, budgets, and other settings fixed while changing the specified assistance. The earlier aggregate has differing coverage and budgets. The recent ablations select

Table 4: Assistance configurations and current reporting status.

Configuration	Status and interpretation
Basic tools	Base interface only.
Candidate filtering	Base interface plus <code>list_legal_folds</code> .
Automatic target feedback	Candidate filtering plus automatic tier-3 feedback; recent ablations.
On-demand target comparison	Candidate filtering plus model-requested tier-3 feedback; earlier runs.

common sample IDs and settings; execution errors and omissions after blocking errors are distinguished in Section 5.2.

4.4.1 TARGET-COMPARISON FEEDBACK

The optional `compare_to_target` tool has three feedback levels, configured for the run rather than selected by the model. These levels are separate from the assistance conditions above and from final verification in Section 4.2. The tool reads the current and target stacks without consulting the reference sequence or changing the state.

Table 5: Implemented target-feedback levels. Their availability does not establish an experimental benefit; comparisons require matched runs.

Level	Information returned
1: Counts	Current and target layer counts, their difference, and a count-based summary.
2: Diagnosis	Level 1 plus aligned layer matches, mismatched ranks, and a diagnosis of count, shape, orientation, or ordering disagreement.
3: Corrections	Level 2 plus suggested corrections for up to twelve mismatched ranks, such as orientation changes or a different target rank for a layer shape.

The richer levels supply more target-specific guidance. Their diagnoses are geometric feedback, not proofs of a valid repair or substitutes for the final state comparator. In particular, a matching layer count does not establish that the sequence is complete or correct.

5 RESULTS

We report the completed Codex runs in the inventory generated on 24 September 2026, separating the broader GPT-5.6 Luna evaluation from the recent ablations. The inventory counts completed attempts, including unsuccessful submissions and stopping-limit terminations. For the ablations, all 360 planned sample-configuration pairs enter the denominator. Execution errors and omissions after blocking errors count as unsuccessful evaluations, with causes marked separately. We retain the first completed attempt per sample and configuration when available; retries do not add denominator entries.

5.1 BROADER EVALUATION

Table 6 reports the earlier GPT-5.6 Luna runs, before the recent ablation cohort. Basic tools provide editing and observation; legal enumeration adds candidate folds. On-demand target comparison is distinct from automatically supplied feedback. Coverage, repetitions, and budgets differ across these earlier conditions, so their aggregate solve rates do not isolate the effect of a tool.

Table 6: Earlier GPT-5.6 Luna evaluation at low reasoning effort. Counts are solved attempts over completed attempts, including repeats across samples and budgets.

Condition	Solved / completed	Solve rate
Basic tools	4/67	6.0%
Legal enumeration	31/272	11.4%
Legal enumeration + on-demand comparison	28/108	25.9%

5.2 TOOL AND REASONING-EFFORT ABLATIONS

The recent study targets ten samples from each of four groups per configuration: easy, medium, hard, and some-layers. It compares GPT-5.6 Luna at low effort with GPT-6 Luna at low and high effort. The three tool conditions are basic tools, legal-fold enumeration, and enumeration with automatic tier-3 target feedback. The run settings are specified in Section 4.3.3. Of 360 planned sample-configuration pairs, 355 have completed episodes, three have observed execution errors without a completed replacement, and two were omitted from the restarted batch after blocking errors. All five count as unsuccessful evaluations under the planned protocol (Appendix B.9).

Table 7: Recent Luna ablations. Cells show solved/evaluated samples; each group has ten intended samples. B: basic tools; L: legal enumeration; A: legal enumeration with automatic tier-3 target feedback. * includes one input-size error; † includes one service prompt rejection; ‡ includes two omissions after blocking errors. All five count as unsuccessful evaluations.

Model	Effort	Tools	Easy	Medium	Hard	Some-layers	Total
GPT-5.6 Luna	Low	B	2/10	0/10	0/10	0/10	2/40
	Low	L	1/10	0/10	0/10	5/10	6/40
	Low	A	4/10	0/10*	0/10	6/10	10/40*
GPT-6 Luna	Low	B	0/10	0/10	0/10	0/10	0/40
	Low	L	1/10	0/10*†	0/10	1/10	2/40*†
	Low	A	3/10	0/10‡	0/10	5/10	8/40‡
GPT-6 Luna	High	B	3/10	0/10	0/10	3/10	6/40
	High	L	3/10	0/10	0/10	7/10	10/40
	High	A	3/10	0/10	0/10	8/10	11/40

For GPT-6 Luna, higher effort improves observed some-layers performance in every tool condition. At high effort, easy-group performance remains 3/10 across conditions; automatic feedback adds one solved some-layers sample over legal enumeration alone. No Luna ablation solves a medium or hard sample. These observations concern ten samples per group and condition.

The inventory also contains small exploratory runs: GPT-6 Sol with legal enumeration solves 1/3 low-effort attempts and 1/1 high-effort attempt; GPT-6 Astra with basic tools solves 1/1 low-effort attempt. Earlier basic-tool runs record 0/7 for GPT-5.6 Sol and 0/1 for GPT-5.6 Terra. One additional completed basic-tool attempt has no recorded model identity and is unsolved. These exploratory runs are outside the Luna ablation set and do not support cross-model rate comparisons.

5.3 SEARCH BASELINE

The separate BFS snapshot in `workspace/RESULTS/results.md` contains 725 attempts: 366/669 solved on easy samples, 1/40 on medium, and 0/16 on hard. The easy solve rate is 54.7%. These attempts include repeated samples and different budgets and are not paired with the recent ten-sample ablations. BFS explores the same enumerated legal action space, but its wall-clock budget differs from the model’s turn and call-timeout limits.

Search-effort statistics and plots are provided in Appendix B.2.

5.4 CREASE-PATTERN DISTANCE

The separate saved-run CP-distance scan covers 1,422 attempts, of which 1,421 have recoverable crease patterns. Mean insertion/deletion distance is 45.454; mean distance normalized by target crease count is 0.553. One attempt is excluded because its state could not be recovered. These averages pool models, baselines, configurations, and repeated samples. This earlier distance snapshot does not provide condition-specific CP distances for the recent ablation table. These attempt-weighted averages pool saved configurations and repeated samples; they are descriptive and do not constitute matched model comparisons. The CP-distance scan may cover more runs than the earlier solve-rate snapshot.

5.5 FAILURE ANALYSIS

The earlier 447-attempt GPT-5.6 Luna termination snapshot records 215 state-cycling stops, 18 repetition stops, 47 turn-budget stops, and 167 completions. A completion label does not imply a solved construction. These counts concern the earlier snapshot; the updated inventory does not tabulate termination reasons for the recent ablations. Repeated exploration without reaching the target indicates ineffective search in those attempts, but the stopping labels alone do not identify its cognitive cause.

Table 8: CP insertion/deletion distance over saved attempts. Means exclude unscored attempts. Normalization divides each distance by its target merged crease count before averaging.

Model	Attempts	Scored	Mean distance	Mean normalized
claude-sonnet-5	5	5	12.800	0.481
codex-cli-chatgpt	1	1	8.000	1.000
deterministic-bfs	725	725	38.810	0.494
deterministic-reverse	203	203	24.212	0.764
gpt-5.6-luna	447	447	67.159	0.538
gpt-5.6-sol	7	7	15.857	0.586
gpt-5.6-terra	1	1	0.000	0.000
gpt-6-astra	1	1	0.000	0.000
unknown	32	31	43.065	0.804
All attempts	1422	1421	45.454	0.553

The medium and hard ablation outcomes are consistent with our hypothesis that longer reference sequences demand more search. They do not isolate sequence length from other geometric differences between groups.

The broader evaluation, recent ablations, BFS snapshot, and CP-distance scan cover different attempt sets. We report their denominators separately. Repeated attempts in the broader inventory are not independent samples, and the pooled counts do not constitute a single controlled comparison.

6 LIMITATIONS

The evaluation concentrates on Luna and BFS, with limited coverage of other models. The recent low- and high-effort ablations use ten samples per group and configuration. Cohort and budget differences are documented in Section 5.5.

Fold-count groups express the hypothesis that longer reference sequences generally require more search. They do not establish minimum solution length or guarantee an ordering of instance difficulty. The action space covers all-layers and selected some-layers simple folds, excluding pre-creasing; the some-layers corpus currently extends to six reference folds. The environment represents zero-thickness flat states and does not model material mechanics or continuous collisions during folding.

Success requires geometric reasoning, state tracking, and planning together. Termination labels identify observed behavior but do not isolate the underlying cause of failure. Revisit and budget limits also constrain the search trajectories that can be observed. Generating new samples reduces reliance on a fixed public test set but cannot guarantee absence of training overlap.

Corpus replay checks share the generation engine and therefore test consistency rather than independent physical validity. The terminal comparator compares current-plane polygons, parity and rank, and does not explicitly match original-sheet region identities, as detailed in Appendix A.6; equal shapes at different original-sheet locations can therefore require a correspondence check this path does not perform.

7 CONCLUSION AND FUTURE WORK

CP2Seq asks a model to recover the folding sequence between a crease pattern and a target folded state, and checks the answer by executing it. Generating an instance costs one forward pass of the engine while recovering a sequence has no known efficient method, which is what lets the corpus grow without annotation and the verifier stay exact. On the easy group a deterministic breadth-first search over the same action set solves 54.7% of its attempts against 20.4% for the best-covered model, and no evaluated model attempt succeeds on the medium or hard groups. Every recorded reasoning-effort setting is low, so these numbers bound what was measured rather than what the models can do.

Two capabilities the benchmark is built to exercise are not yet trained against it: deciding when to abandon a partial sequence, and deciding which candidate folds to discard unexamined. The 48.1% of attempts that end in state cycling (Section 5.5) is where both are visible, and neither is addressed by a larger query budget. We report the gap and leave its closure, whether by search-aware prompting, by tool design, or by training against the verifier, to future work. The remaining assistance conditions, higher reasoning-effort runs, and matched budgets across configurations are the measurements that would make those comparisons controlled.

8 LICENCE AND AVAILABILITY

The corpus generator, the environment and verifier, both corpus checks, the state comparator and the evaluation harness are released under the **BSD 3-Clause** licence.

Code and data are released on publication; links are omitted here.

The artifacts carry no personal data, no human-subject data and no scraped content. Every sample is synthesised by folding a square, so no licensing question attaches to the geometry and no attribution is owed to any origami designer. Flat-Folder’s `examples/instagram/` set, the one external corpus this paper measures against, is not redistributed; the scope measurement of Section 6 was computed against it in place and is reported as a number.

9 USE OF LARGE LANGUAGE MODELS

Language models are evaluated as agents in the folding task. AI assistants were also used for implementation, literature-search assistance, and manuscript drafting and editing. Claude Code was used for coding assistance. AI-generated figure drafts served as composition references; dataset geometry and experimental results must be rendered from the corresponding artifacts. The authors are responsible for the manuscript and released implementation.

ACKNOWLEDGMENTS

Withheld for anonymous review.

REFERENCES

- Hugo A. Akitaya, Jun Mitani, Yoshihiro Kanamori, and Yukio Fukui. Generating folding sequences from crease patterns of flat-foldable origami. In *ACM SIGGRAPH 2013 Posters*, pp. 1. Association for Computing Machinery, 2013. doi: 10.1145/2503385.2503407.
- Hugo A. Akitaya, Erik D. Demaine, and Jason S. Ku. Simple folding is really hard. *Journal of Information Processing*, 25:580–589, 2017. doi: 10.2197/ipsjip.25.580.
- Hugo A. Akitaya, Erik D. Demaine, and Jason S. Ku. Computing flat-folded states. In Guoxing Lu, Zhong You, and Michael Assis (eds.), *Origami8, Volume III*, pp. 201–219. Springer, 2026. doi: 10.1007/978-981-96-6561-7_14.
- Esther M. Arkin, Michael A. Bender, Erik D. Demaine, Martin L. Demaine, Joseph S. B. Mitchell, Saurabh Sethia, and Steven S. Skiena. When can you fold a map? *Computational Geometry*, 29(1):23–46, 2004.
- Marshall Bern and Barry Hayes. The complexity of flat origami. In *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 175–183, 1996.
- Jason S. Ku. Flat-Folder: A crease pattern solver, 2022. URL <https://github.com/origamimagiro/flat-folder>. Software repository, accessed 25 September 2026.
- OpenAI. Codex CLI. Open-source software, version 0.155.1, 2026. URL <https://github.com/openai/codex/tree/be2951ea34f0d295ed0becf97079f92fa5f6950e>. Release tag rust-v0.155.1; accessed 25 September 2026.
- Ryan Spencer, Roey Yaari, Ritvik Vemavarapu, Joyce Yang, Steven Ngo, and Utkarsh Sharma. GamiBench: Evaluating spatial reasoning and 2D-to-3D planning capabilities of MLLMs with origami folding tasks. *arXiv preprint arXiv:2512.22207*, 2025. URL <https://arxiv.org/abs/2512.22207>.
- Rui Xu, Dakuan Lu, Zicheng Zhao, Xiaoyu Tan, Xintao Wang, Siyu Yuan, Jiangjie Chen, and Yinghui Xu. OrigamiSpace: Benchmarking multimodal LLMs in multi-step spatial reasoning with mathematical constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://neurips.cc/virtual/2025/poster/115182>.

A FOLDING MODEL AND VERIFICATION

A.1 FLAT-FOLDED STATES

A state represents the positions of paper regions in the plane, their orientations, and their layer ordering. Original-sheet coordinates identify each region and preserve its connectivity to other regions as folds change its current position. Overlapping regions can occupy the same planar area while remaining distinct layers. Their original-sheet identities therefore matter both when validating a move and when comparing the completed construction with the target. Layer ordering is also a central component of computational treatments of flat-folded states (Akitaya et al., 2026).

A.2 SUPPORTED ACTIONS

The simple-fold literature distinguishes operations by the layers they move (Arkin et al., 2004). Our all-layers action selects the paper on one side of a fold line throughout the stack. Our some-layers action restricts that selection to a contiguous run from the top or bottom. An action specifies the line in current folded coordinates, the moving side, the layer selection, and whether the moving paper folds over or under the stationary paper. The selected paper is reflected across the line, with the corresponding orientation and stack-order updates. In the implemented some-layers model, a top run folds over and a bottom run folds under.

These action models specify the permitted moves. They are distinct from the assistance conditions, which specify the observations, feedback, and tools available to the model. In particular, access to candidate enumeration changes the assistance provided without defining a new folding operation.

A.3 CONNECTIVITY AND TEARING

A move must preserve the connections between regions of the original sheet. If moving paper is attached to stationary paper along a segment away from the proposed hinge, the operation would separate connected paper and is rejected with a `would-tear` response. The check uses original-sheet connectivity together with the proposed motion; overlap in the current plane alone cannot establish whether two regions are attached. Rejection leaves the current state unchanged.

Tearing is one action constraint rather than a complete model of physical feasibility. The engine also checks the action parameters and compatibility with the target crease pattern. It represents zero-thickness flat states and does not test continuous collisions during the folding motion. Step-level legality and final target agreement are separate checks: a sequence of legal actions can still produce the wrong construction.

A.4 DEPTH AND SEARCH DIFFICULTY

The recorded sequence provides a reachable target, but need not be a shortest solution. Reference depth therefore describes generation rather than minimum planning horizon. Likewise, the number of legal moves and the probability that a random proposal is legal are different quantities: a larger proposal space can lower the latter without reducing the former. We group samples by reference fold count, hypothesizing that longer sequences generally require more search. These groups describe expected difficulty, not a measured minimum solution length.

A.5 COMPUTATIONAL COMPLEXITY

The sequence-generation method of Akitaya et al. (2013) and the hardness proofs concern different contributions. For a precise hardness result, Akitaya et al. (2017, Theorem 3) prove that deciding simple foldability of square paper with assigned creases at multiples of 45° is strongly NP-complete in their some-layers and all-layers models. The input is an arbitrary crease pattern; the question is whether any permitted sequence folds it. Strong NP-hardness persists when numerical magnitudes are polynomially bounded.

The reduction starts from 3-PARTITION: divide integers into triples with a common sum. In the orthogonal-paper construction (Theorem 1), pleating a staircase encodes choices of integers. A bar must pass through successive cage slots; the geometry forces each group of choices to contain three integers with the required sum. A partition therefore yields a folding sequence, and a complete folding yields a partition. Theorem 3 embeds this construction in square paper using additional creases that enforce the preparatory folds. The construction has polynomial size, and a proposed folding order can be checked in polynomial time. This is a proof sketch of the cited result, not a new reduction.

For CP2Seq, each target is generated by a recorded legal sequence, so reachability is promised rather than decided. The model must recover a sequence that also matches a supplied terminal state, under the implemented action rules and numerical tolerances. The theorem above does not by itself prove hardness of this

promised reconstruction problem or of our sample distribution. Its relevance is that general sequence selection can encode combinatorial choices even when individual actions can be checked efficiently. In an episode, the practical search burden depends on the available actions, solution depth, and the number of unsuccessful branches explored; fold count alone does not determine that burden.

A.6 COMPARISON IMPLEMENTATION

The engine compares creases in original-sheet coordinates after grouping and merging collinear intervals by assignment. Interval endpoints are matched within 2×10^{-6} ; coordinate canonicalization is part of preprocessing and does not make the comparison tolerance-free.

The shared terminal comparator first checks layer counts. It considers direct alignment and turnover, with turnover reversing layer order and face parity. Correspondences between vertices of the first layer propose global alignments; a candidate alignment is accepted only if every layer polygon and parity matches its target rank. Polygon matching uses a positional tolerance of 2×10^{-6} , short-edge cleanup at 10^{-7} , and a collinearity threshold of 10^{-9} .

Table 9 collects every numerical tolerance the reported results depend on. Each result record carries the values it was scored under, so a re-scoring can be distinguished from a comparison at the same settings.

Table 9: Numerical tolerances used by the reported checks. No check in this pipeline is tolerance-free.

Value	Where used	What it admits
2×10^{-6}	Crease comparator; terminal comparator	Interval endpoint and polygon vertex positions, after an arbitrary global isometry.
10^{-7}	Terminal comparator	Short-edge cleanup before polygon matching.
10^{-9}	Terminal comparator	Collinearity threshold for vertex removal.
2×10^{-6}	CP edit distance	Interval endpoint agreement.
2.44×10^{-4}	CP edit distance	Line quantization when grouping collinear intervals.
10^{-5}	CP edit distance	Gap below which two intervals on one line are merged.

The shared implementation represents current-plane polygons, parity, and rank. It does not explicitly compare original-sheet region identities. Any identity-aware offline comparator must be distinguished from this path, and the comparator version used for each reported result must be recorded. In particular, equal shapes at different original-sheet locations can require a stronger correspondence check than current-plane geometry alone.

The repository contains comparison and verification tests; executed test results and the exact comparator revision must accompany any reported validation.

A.7 ACTION REJECTION CONDITIONS

Table 10: Geometric and action constraints checked by the implementation. Rejected edits preserve the previous state.

Code	Condition
would-tear	A moving region and a stationary region share an original-sheet edge whose current position is not on the fold line. Reflecting one side would separate the join.
no-crease	A nonempty moving selection produces no crease segment in the paper. This condition is checked separately from tearing.
nothing-to-move	The selected half-plane contains no selected paper, or the requested partial selection contains no layers.
direction-impossible	The action requests folding a top run under the stack or a bottom run over it, contrary to the supported layer-selection rule.

The independent Flat-Folder wrapper uses layer-order constraints, including taco-taco, taco-tortilla, tortilla-tortilla, and transitivity constraints (Akitaya et al., 2026; Ku, 2022). It tests whether the pattern admits a flat-folded state, not whether the agent’s sequence reaches the supplied target. It is not invoked automatically for every proposed action.

B SOFTWARE AND SUPPLEMENTARY EXAMPLES

B.1 SOFTWARE VERSIONS

The inspected Codex CLI installation is version 0.155.1 (OpenAI, 2026). Its upstream release tag `rust-v0.155.1` resolves to commit `be2951ea34f0d295ed0becf97079f92fa5f6950e`. This identifies the client software, not the model weights or server implementation. The saved run configurations record the executable path but do not record its version or binary hash; the installation was checked on 25 September 2026.

B.2 BREADTH-FIRST SEARCH ANALYSIS

A fold is specified by a line in current-plane coordinates together with a layer selection, and both are drawn from small finite sets. Candidate lines are read off the crease pattern itself: the distinct lines through its mountain and valley edges, carried forward through each distinct face transform into current coordinates, then deduplicated. On a box-pleated pattern thousands of crease edges collapse to a few dozen lines. Layer selections are not arbitrary subsets either, because a partial fold must be a contiguous run at one extremity of the stack; the selections are the whole stack, the top k , and the bottom k , which is about $2N$ for N layers rather than 2^N , with the over or under direction forced for partials. Each surviving candidate is judged by the same function the `add_fold` tool calls, so a listed fold cannot be one the tool would then reject, and it is dropped if it fails any material check of Section 4.1 or would crease the sheet where the target pattern carries no crease.

Across 73,750 enumerations recorded in the saved runs, this evaluates a mean of about 2,010 candidate actions per state and returns 3.51 of them, a ratio of approximately 570 between evaluated and returned candidates. The baseline expands with the same enumerator the model is given, tests goals with the same terminal comparator that decides `solved`, reads no reference sequence, and runs under a per-sample wall-clock budget with caps on states expanded and depth.

The supplementary cost model approximates search time as the product of two exponential factors: the frontier grows with the branching factor, and the cost of expanding one node grows with depth as well, because one enumeration evaluates four candidates per crease line per facet and a fold of all layers at most doubles the facets. Fitting both from the saved searches gives a per-fold multiplier of 3.9 on the easy group, 4.6 on medium and 12.4 on hard (Figure 4). These fitted curves extrapolate beyond the observed depths; they are not measured solution times or lower bounds. The hard-group fit uses only two samples.

B.3 EVALUATION PIPELINE

B.4 TOOL SCHEMAS AND PROMPT

The following source listings document the available arguments and the folding prompt. Tool exposure varies by run configuration; the full schema listing is not a claim that every run exposed every tool.

```
"""Schemas supplied to the native Qwen3.5 tool-aware vision renderer."""
import json
```

```
def tool(name, description, properties=None, required=None):
    return {"type": "function", "function": {"name": name, "description": description,
        "parameters": {"type": "object", "properties": properties or {},
            "required": required or [], "additionalProperties": False}}}
```

```
TOOLS = [
    tool("add_fold", "Append a 180-degree fold. Choose angle_index OR angle_degrees. Partial top/bottom runs must not tea
        "angle_index": {"type": "integer", "enum": [0, 1, 2, 3]},
        "angle_degrees": {"type": "number", "description": "Line angle counterclockwise from +x; offset uses unit normal
        "selection_mode": {"type": "string", "enum": ["all", "top", "bottom"]},
        "layer_count": {"type": "integer", "minimum": 1, "description": "Required for top/bottom, omit for all."},
        "offset": {"type": "number"}, "move_positive": {"type": "boolean"},
        "over": {"type": "boolean"}, ["offset", "move_positive", "over"]},
    tool("remove_fold", "Remove 1-based fold; replay the rest transactionally.",
        {"step": {"type": "integer", "minimum": 1}}, ["step"]),
    tool("go_to_step", "Keep the first N folds; zero clears the sequence.",
        {"step": {"type": "integer", "minimum": 0}}, ["step"]],
```

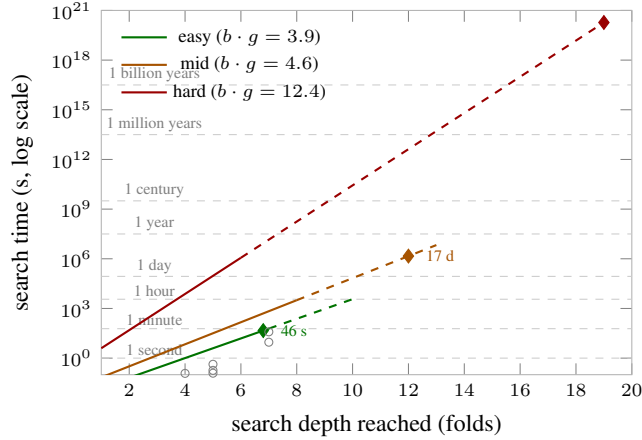


Figure 4: **Breadth-first search time against search depth, by difficulty group.** The vertical axis is logarithmic, so a straight line is exponential growth and its slope is $\log(b \cdot g)$. Curves are $t(d) = c_0(b \cdot g)^d$, the product of two exponentials: the frontier grows as b^d for a deduplicated branching factor b , measured as generated over expanded during search, while the cost of expanding one node grows as g^d , because one enumeration evaluates $4 \times (\text{distinct crease lines}) \times (\text{facets})$ candidates and a fold of all layers at most doubles the facets. g and c_0 are fitted by least squares on per-depth enumeration timings. Each curve is solid only where that group was observed and dashed beyond, and spans only the depths its own corpus reaches. Diamonds are each group’s mean reference depth; the hard group’s sits at 10^{20} seconds, which is why the axis runs that far. Circles are observed solves. b is fitted from budget-exhausted runs: 7 over 3 samples for easy, 12 over 3 for medium, and 8 over 2 for hard, across 5 time budgets. The hard extrapolation rests on two samples. Extrapolated values are model projections, not measured solution times or lower bounds.

Group (reference folds)	Median states expanded	Solved / attempts
Easy (4–10)	65	366/669
Medium (11–13)	1,051.5	1/40
Hard (14–19)	532	0/16

Figure 5: Observed BFS search effort in the saved aggregate. Bar lengths use a linear scale and show median states expanded across attempts, including unsuccessful attempts. Budgets and repeated samples are not standardized by this aggregate; 358 of 725 attempts terminated in timeout. The lower hard-group median does not imply that hard instances require less search. Reference depth is not minimum solution length.

```

848 tool("restore_revision", "Restore a returned revision, including abandoned branches.",
849       {"revision": {"type": "integer", "minimum": 0}}, [{"revision"}]),
850 tool("get_images", "Render top, two oblique, X-ray, and exploded-stack PNGs at a completed step.",
851       {"step": {"type": "integer", "minimum": 0}}),
852 tool("get_state", "Inspect the current sequence and bottom-to-top layer polygons."),
853 tool("list_legal_folds", "List the folds that are legal in the current state and on-target for the CP. "
854       "Read-only. Every listed action is accepted verbatim by add_fold while the state is unchanged.", {
855       "max_results": {"type": "integer", "minimum": 1},
856       "selection_filter": {"type": "string", "enum": ["any", "all", "top", "bottom"]},
857       "include_rejected": {"type": "boolean"}}),
858 tool("compare_to_target", "Compare the current stack with the target folded state. Read-only. "
859       "Detail is fixed by the run's configured tier, not by you. Uses the target state only; "
860       "the reference fold sequence is never consulted."),
861 tool("finish", "End this episode and evaluate the current sequence."),
862 ]
863
864 # The Codex loop exposes only a subset in its baseline condition; the enumerator changes
865 # what the benchmark measures, so it must be requested explicitly. See TODO.md.
866 ENUMERATION_TOOLS = ("list_legal_folds",)
867 COMPARE_TOOLS = ("compare_to_target",)
868
869 def _whole_stack_only(tools):
870     """Strip partial folds from the schemas, so nothing offers what ToolSession will refuse.

```

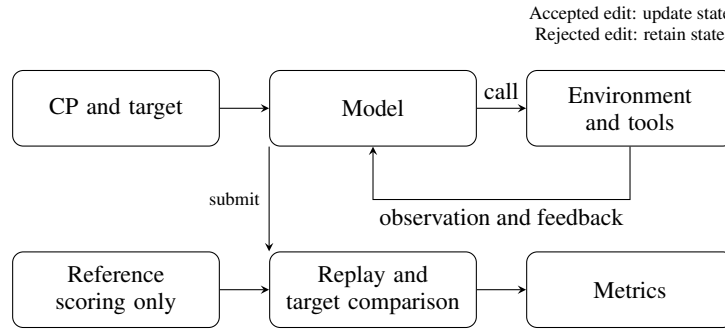


Figure 6: Evaluation pipeline. The model edits and inspects its current construction. The reference sequence is withheld from the model and used only for scoring.

```

Two edits, and both are needed. add_fold loses selection_mode and layer_count, or the
model can request a run the simulator then rejects; list_legal_folds loses
selection_filter, whose remaining values would all be lies once the run forces "all".
"""
out = []
for t in tools:
    name = t["function"]["name"]
    if name not in ("add_fold", "list_legal_folds"):
        out.append(t)
        continue
    copy = json.loads(json.dumps(t))
    props = copy["function"]["parameters"]["properties"]
    for key in ("selection_mode", "layer_count", "selection_filter"):
        props.pop(key, None)
    if name == "add_fold":
        copy["function"]["description"] = (
            "Append a 180-degree fold of the whole stack. Choose angle_index OR angle_degrees.")
    out.append(copy)
return out

def tools_for(mode, compare_tier=0, action_space="any"):
    """The action set for one experimental condition.

    compare_to_target appears only when a tier is configured, so the default arms are
    byte-identical to every run recorded before the comparison tool existed. action_space
    defaults to "any" for the same reason: nothing changes unless a run asks for it.

    action_space is a property of the corpus, not a preference. On release/all-layers every
    one of the 4180 reference folds is whole-stack, so partial folds cost 3-87x in search and
    buy nothing; on some-verified-d3 and some-generated-d6 they are what 14% and 34% of the
    samples require. Whichever is chosen must be given to the search arm too, or the
    comparison measures the setting rather than the solver.
    """
    if mode == "base":
        chosen = [t for t in TOOLS if t["function"]["name"] not in ENUMERATION_TOOLS]
    elif mode == "legal-folds":
        chosen = list(TOOLS)
    else:
        raise ValueError(f"Unknown tool mode: {mode}")
    if not compare_tier:
        chosen = [t for t in chosen if t["function"]["name"] not in COMPARE_TOOLS]
    if action_space == "all-layers":
        chosen = _whole_stack_only(chosen)
    elif action_space != "any":
        raise ValueError(f"Unknown action space: {action_space}")
    return chosen

```

Recover a folding sequence from the supplied crease pattern (CP) and final folded state. The sequence starts empty. Use one tool call at a time, then inspect its result before proposing the next fold. Do not return an entire sequence in prose. The environment writes your accepted actions into seq.json.

You receive only geometry and images of the CP, final target, and current state. No reference folding sequence is supplied. All folds are flat 180-degree simple

918 folds. Select all layers (default), or a contiguous run from the top or bottom.

919 Coordinates are x right, y up. Fold axes are in CURRENT folded coordinates:

```
920 | angle_index | fold line | positive side |
921 | --- | --- | --- |
922 | 0 | y = offset | y > offset |
923 | 1 | y = x + offset | y - x > offset |
924 | 2 | x = offset | x > offset |
925 | 3 | x + y = offset | x + y > offset |
```

926 For arbitrary line angles, supply angle_degrees instead of angle_index, never
 927 both. With theta counterclockwise from +x, the line is
 928 $-\sin(\theta)x + \cos(\theta)y = \text{offset}$; offset is signed perpendicular distance.
 929 move_positive selects the greater-than side. The folding motion remains 180 degrees.
 930 selection_mode is all (default), top, or bottom. Top/bottom requires layer_count;
 931 omit layer_count for all. Top must fold over; bottom must fold under.
 932 The engine tracks original-sheet connectivity. A partial fold returns would-tear
 933 if selected paper is attached to stationary paper away from the proposed hinge.
 934 For example, selecting three of four layers may tear a paired flap. This check
 935 runs before target-CP checking and leaves state unchanged on rejection. Inspect
 936 sheet_polygon coordinates and exploded views when choosing a connected run.

937 move_positive selects which half-plane moves. over=true puts the reflected moving
 938 stack above the stationary stack; false puts it below. A flipped stack reverses
 939 its internal order. Offset is not a normalized distance for diagonal axes.
 940 The tool rejects extra creases or incompatible mountain/valley assignments and
 941 leaves the state unchanged on error. This is a zero-thickness geometric engine;
 942 it does not check continuous collisions during motion.

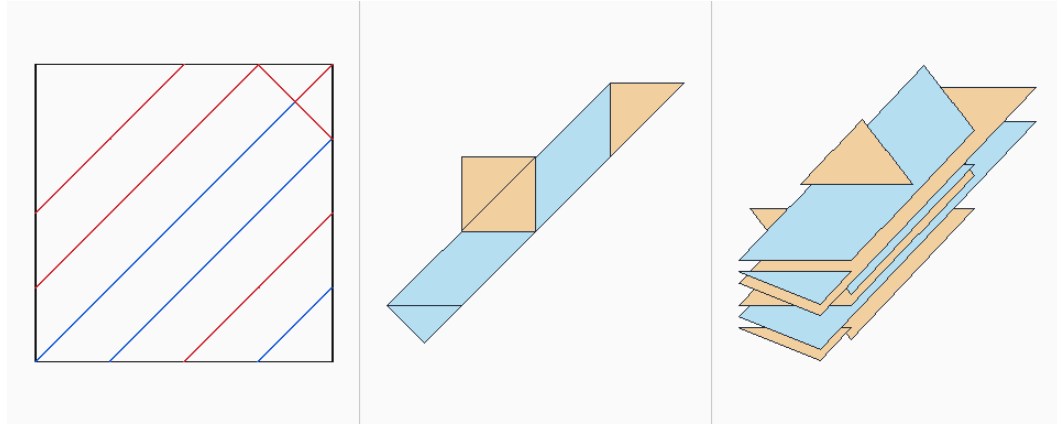
943 add_fold appends ONE fold. remove_fold removes a 1-based fold and replays the
 944 remaining actions; an invalid replay leaves the sequence unchanged. go_to_step
 945 keeps the first N folds (0 empties the sequence). restore_revision restores a
 946 previous tool state, even after changing branches; revision 0 is the empty sheet.
 947 These editing operations create new revision IDs. get_state returns your current
 948 sequence and geometry. get_images returns PNG views of any completed step in the
 949 current branch without changing it. Use the returned revision IDs when backtracking.

950 Images: CP red = mountain, blue = valley, dark boundary = paper edge. Top and
 951 two oblique views show paper shape with a small illustrative layer separation.
 952 Oblique views look down from opposite directions, never exactly edge-on. X-ray
 953 is a top projection: more overlapping layers produce darker pixels. Each view
 954 is fitted independently; use geometry for coordinates and scale, not pixel size.
 955 Exploded views separate layers upward for inspection: layer 1 is the bottom,
 956 and p0/p1 labels give face parity. Separation is illustrative, not paper thickness.
 957 Initial images include the target's exploded view; feedback images include the
 current state's exploded view after edits.

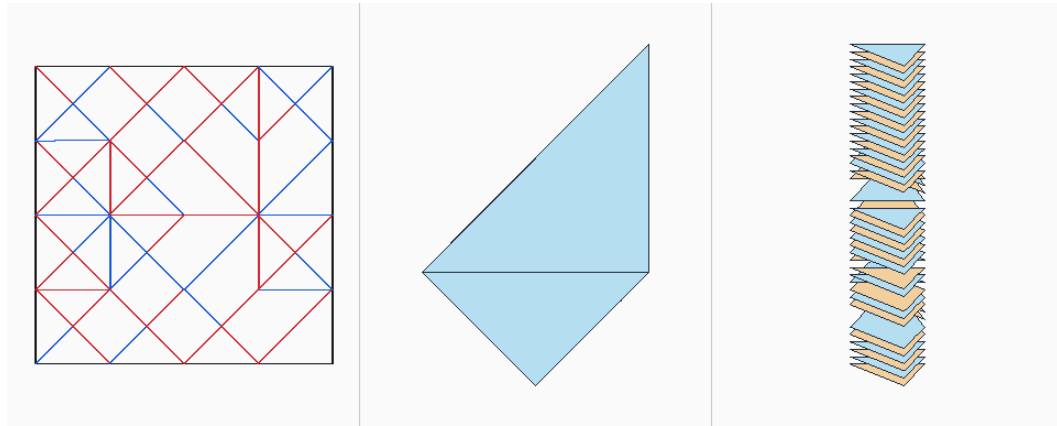
958 Try to reproduce both the CP and the supplied target, including face orientation
 959 and layer order. Terminal matching accepts any global translation and any rotation
 960 angle, including 30 degrees. It also accepts turning the whole model upside down:
 961 reflect the geometry, reverse the bottom-to-top stack, and flip every face parity.
 962 These operations can be combined. One common rigid transformation must match all
 963 layers; arbitrary layer rearrangements, scaling, or independently moving layers
 964 do not count. CP matching still uses the supplied original-sheet coordinates and
 965 mountain/valley assignments. Call finish when you are done. This ends the episode and evaluates
 966 current sequence; it does not automatically claim success.

968 B.5 DATASET GALLERY AND REFERENCE SEQUENCES

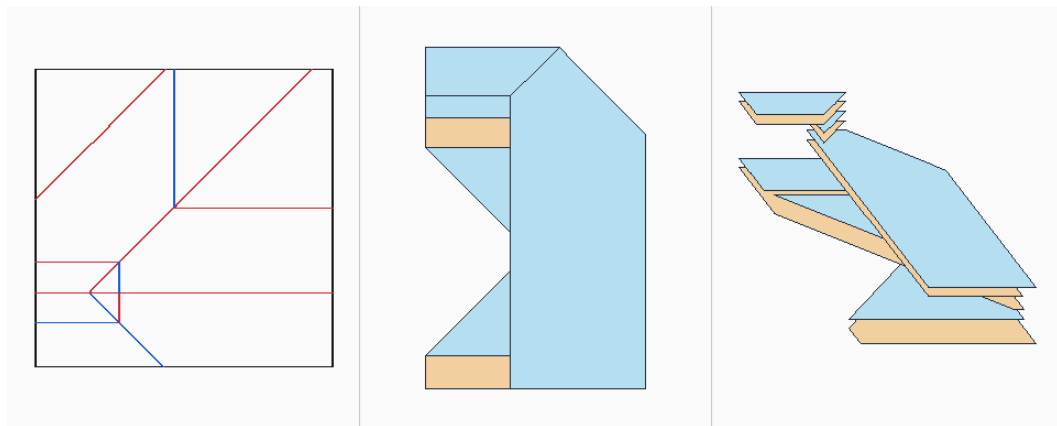
970 Each rendered panel contains the crease pattern, top view, and an oblique view with illustrative layer separation.
 971 Layer separation is a visualization aid, not simulated paper thickness. all-layers/easy-0001 (all-layers, 4 reference folds)



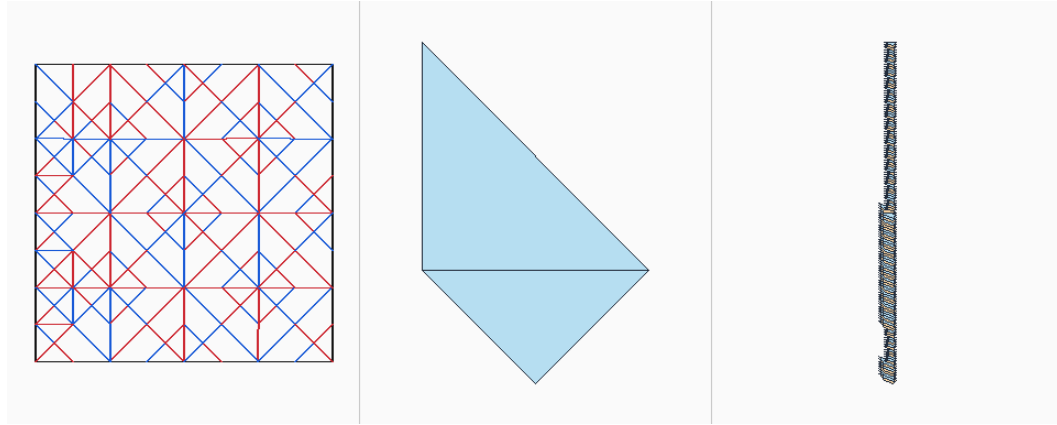
all-layers/mid-0004 (all-layers, 11 reference folds)



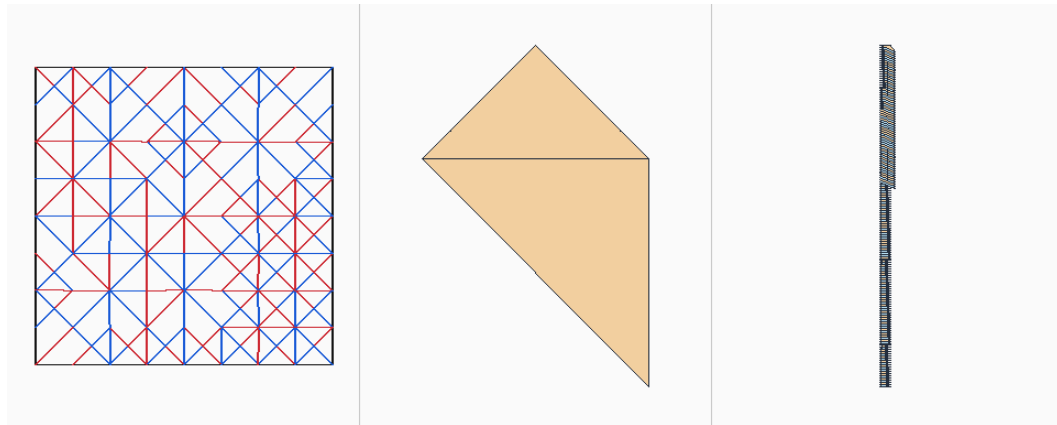
some-generated-d5/layers-0001 (some-layers, 5 reference folds)



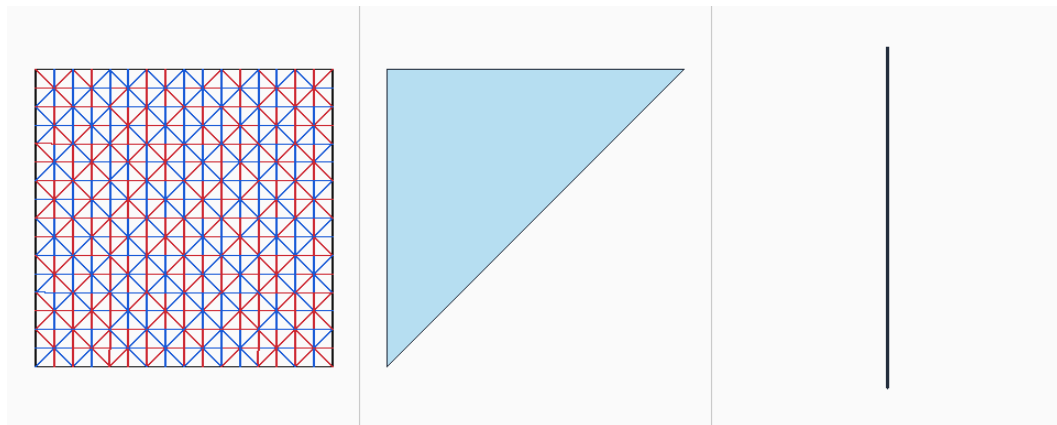
all-layers/hard-0004 (all-layers, 15 reference folds)



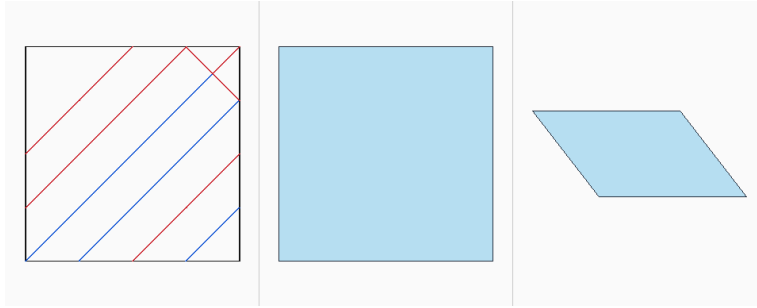
all-layers/hard-0005 (all-layers, 15 reference folds)



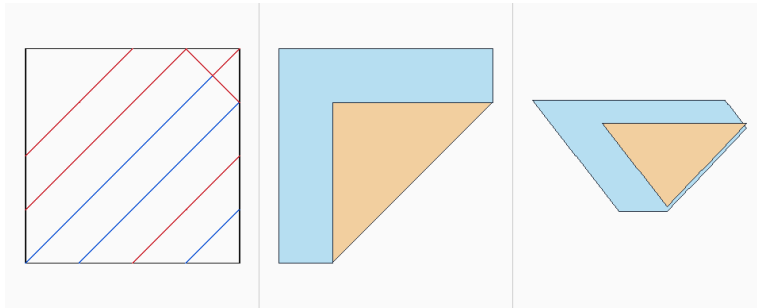
all-layers/hard-0006 (all-layers, 15 reference folds)



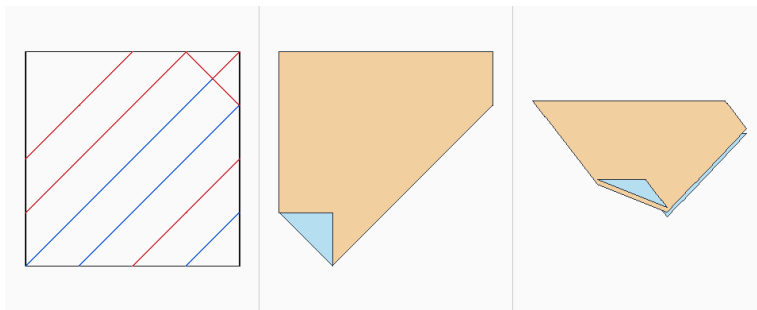
all-layers/easy-0001 (all-layers, 4 reference folds) Stored state 0



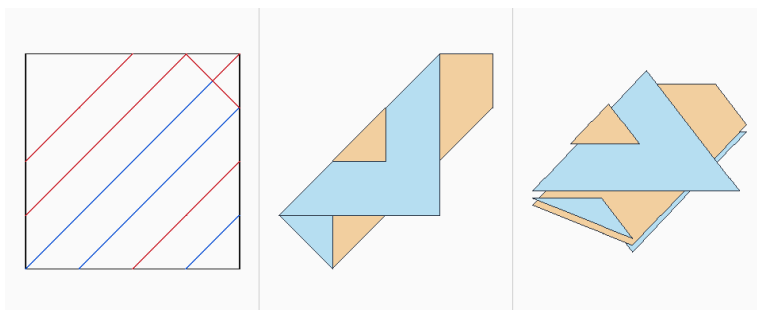
Stored state 1



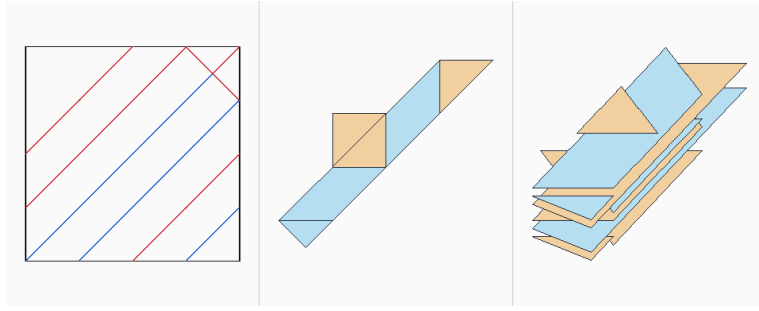
Stored state 2



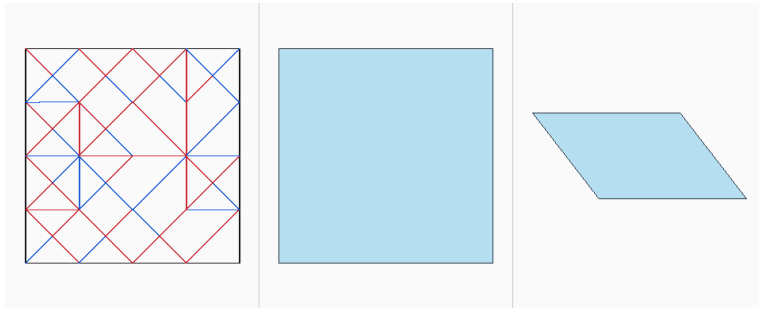
Stored state 3



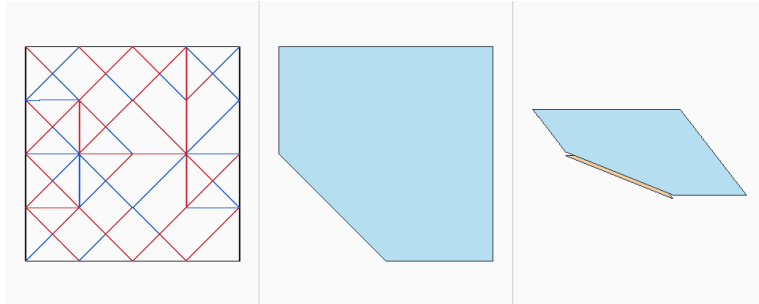
Stored state 4



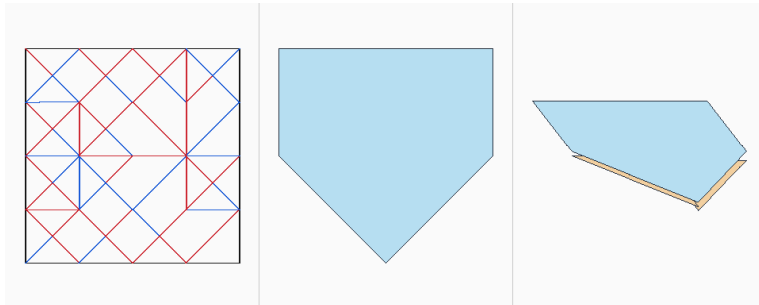
all-layers/mid-0004 (all-layers, 11 reference folds) Stored state 0



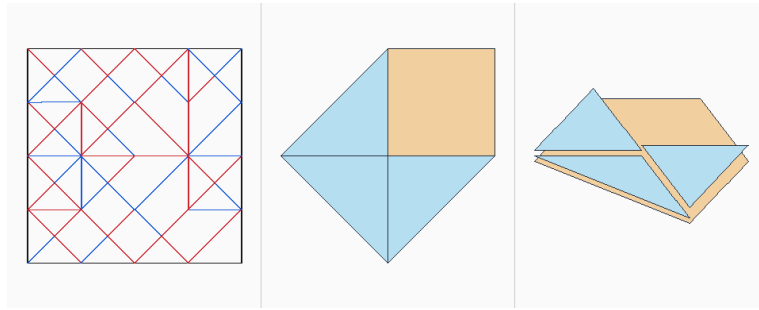
Stored state 1



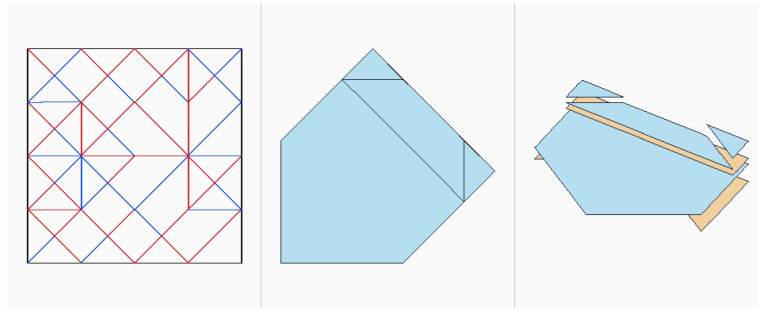
Stored state 2



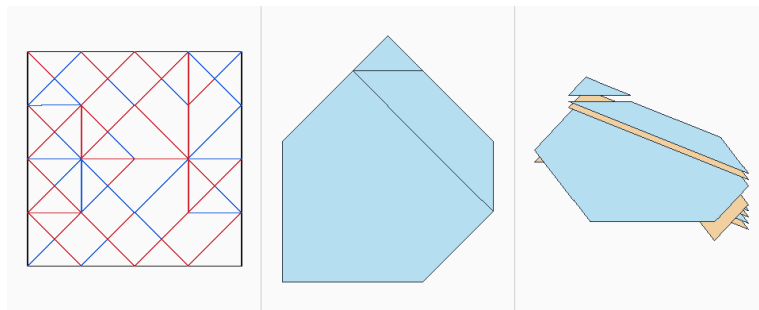
Stored state 3



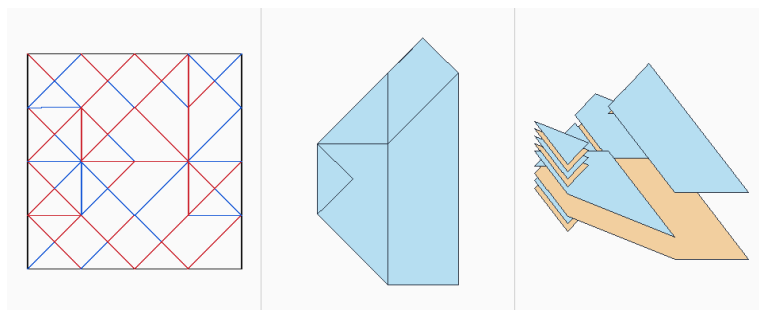
Stored state 4



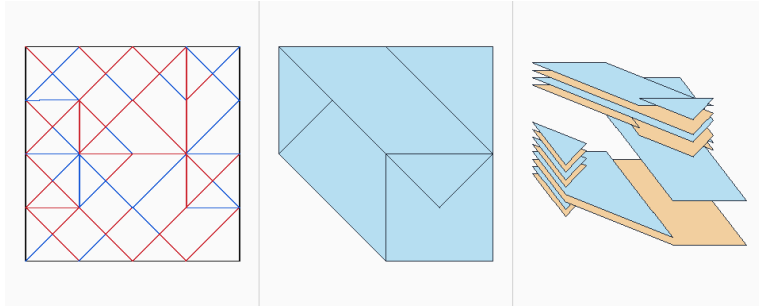
Stored state 5



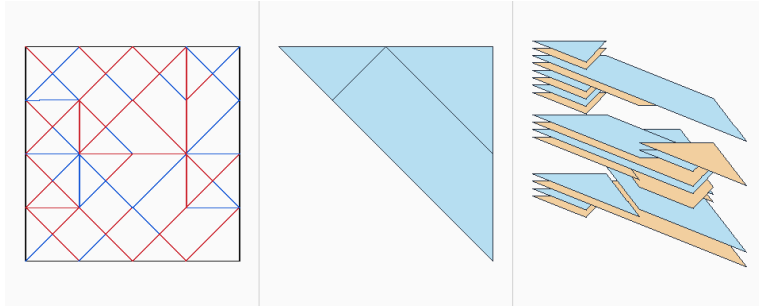
Stored state 6



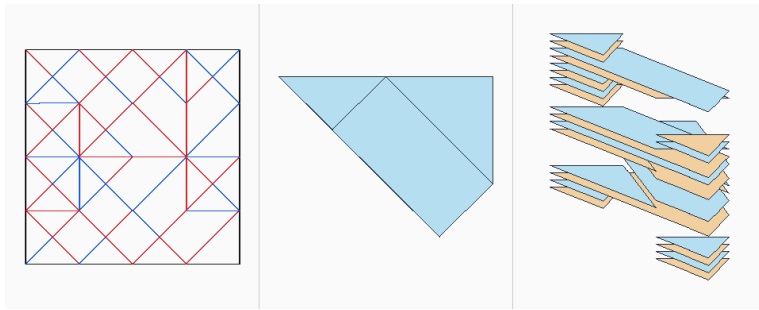
Stored state 7



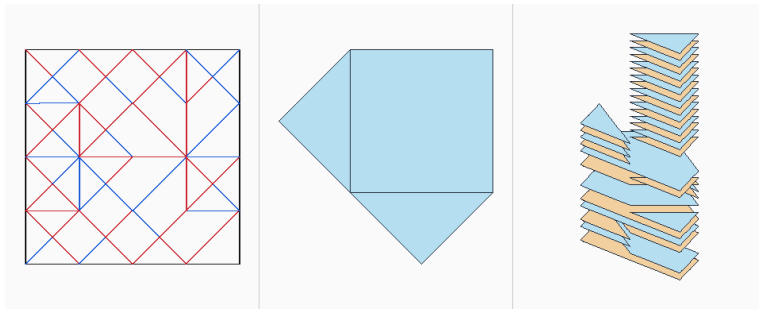
Stored state 8



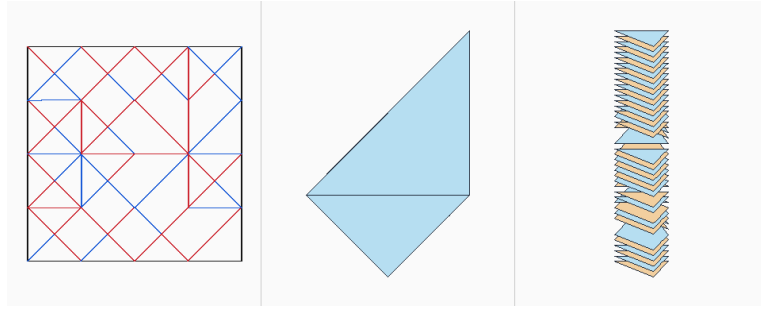
Stored state 9



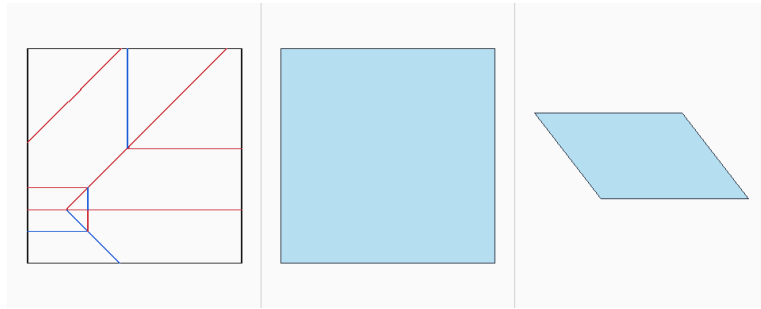
Stored state 10



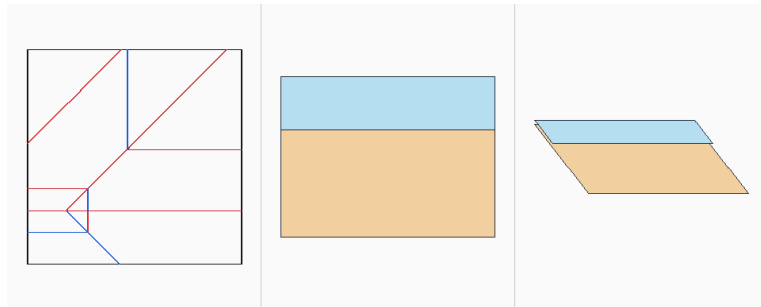
Stored state 11



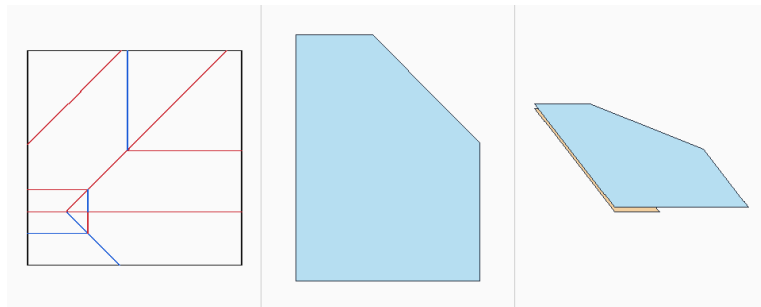
some-generated-d5/layers-0001 (some-layers, 5 reference folds) Stored state 0



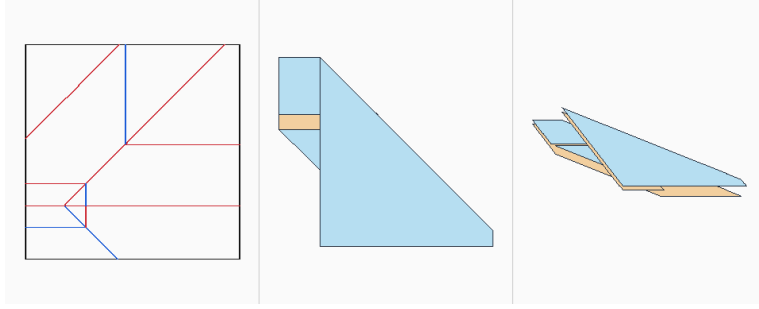
Stored state 1



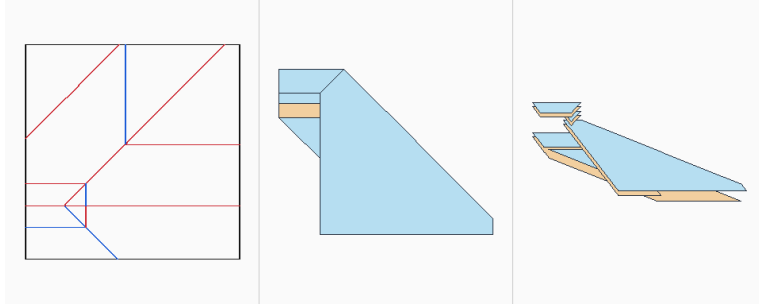
Stored state 2



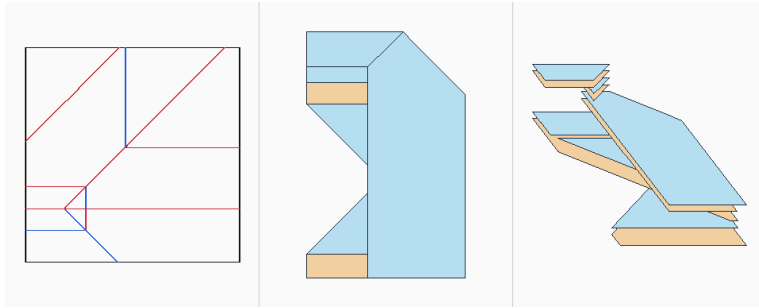
Stored state 3



Stored state 4



Stored state 5



B.6 TRACE-ANALYSIS INTERFACE

We provide a trace-inspection interface for browsing saved runs, replaying recorded folding trajectories, and examining model actions alongside tool feedback and rendered states. This supports qualitative analysis of successful construction, repeated exploration, and unsuccessful termination. The interface presents recorded artifacts; it does not establish the model’s internal reasoning process. A command-line trace viewer additionally supports filtering by task, trial, session, role, and event type.

Figures 7 and 8 show the dataset browser and trace viewer, respectively. These interfaces support inspection of reference sequences and recorded model episodes.

B.7 TOOL LOADING AND CONFIGURATION

The schema builder selects the tools exposed in each run. Basic mode excludes candidate enumeration; filtered mode includes it. Target comparison is added only when a comparison level is configured. For an all-layers action model, partial-layer selection arguments are removed from the exposed action schemas. The run configuration therefore records tool mode, comparison level, and action model separately.

B.8 REJECTION FEEDBACK

Figure 9 illustrates the geometric evidence associated with a `would-tear` response using the shared verification fixtures. The comparison holds the initial state and hinge fixed and changes the selected layers. The export script records both engine responses and checks that rejection preserves the state.

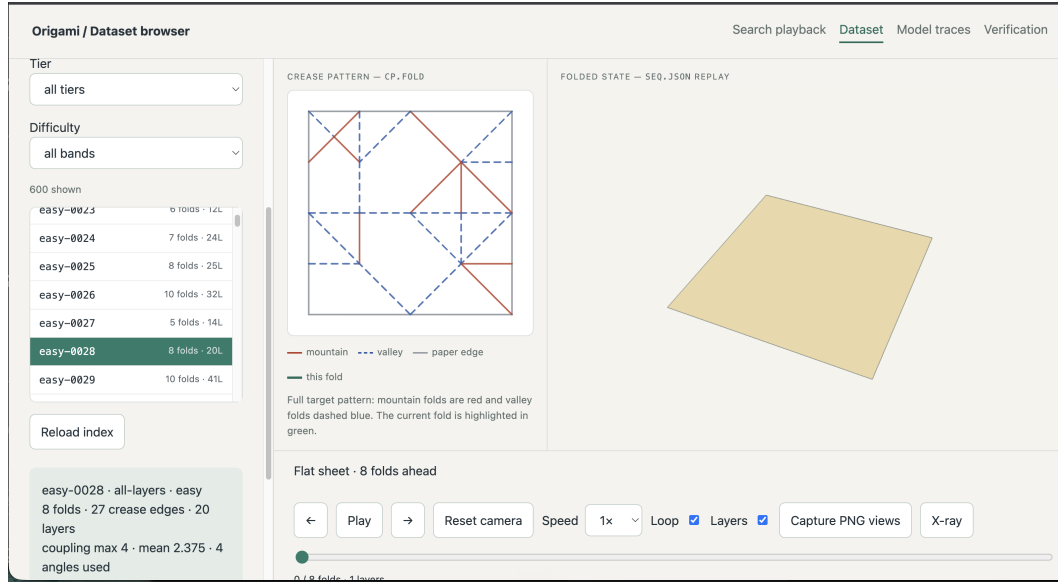


Figure 7: Dataset browser with `easy-0028` selected. The interface displays sample metadata, the target crease pattern, and reference-sequence playback controls. The replay is at step zero: the right panel shows the unfolded sheet, not the target folded state.

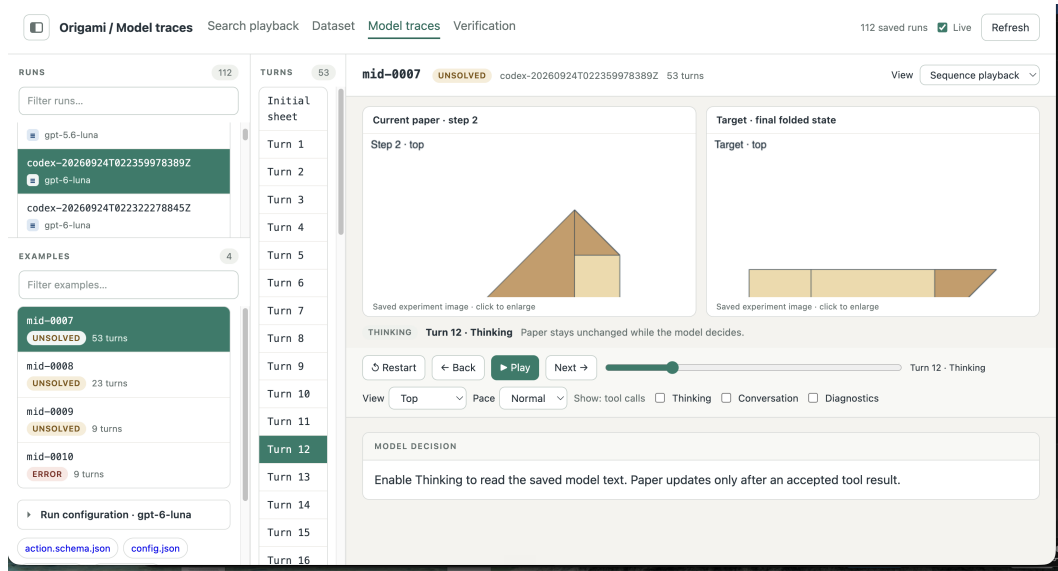


Figure 8: Trace viewer for a saved GPT-6 Luna attempt on `mid-0007`. Run and turn navigation appear beside the current and target states. The selected episode is marked unsolved; the displayed frame is an intermediate state at turn 12, not its terminal construction. Playback controls allow inspection of saved actions and feedback.

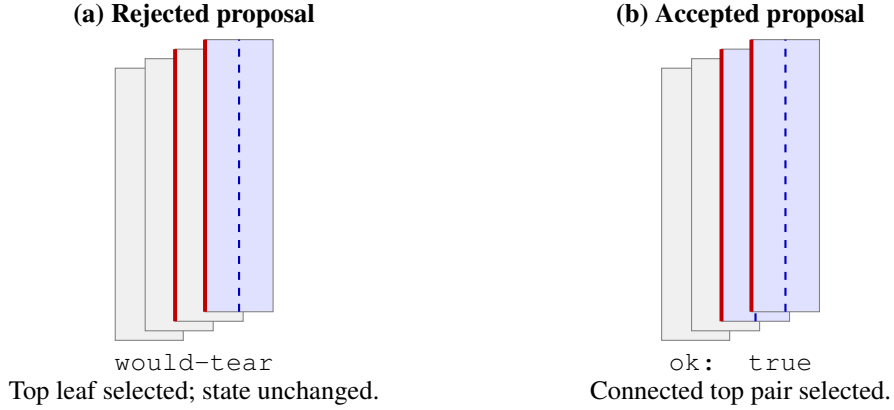


Figure 9: Rejection feedback and a legal alternative from the same state. Dashed blue lines mark the proposed hinge; red segments mark the shared material edge identified by the tearing diagnostic. Blue shading identifies selected layers. Selecting only the top leaf would separate it from its connected neighbour away from the hinge; selecting the connected top pair permits the fold. Both panels show the pre-action state, with layers separated only for visibility. This is a visualization of the verification fixtures and structured feedback, not an image returned to the model or a recorded model attempt.

B.9 PROTOCOL FAILURE ACCOUNTING

All 360 planned Luna ablation cases enter the solve-rate denominator. Of these, 355 have completed episodes. Two cases end at the request input-size limit: `mid-0006` for GPT-5.6 Luna low with automatic feedback and GPT-6 Luna low with legal enumeration. GPT-6 Luna low with legal enumeration on `mid-0010` ends in a service prompt rejection. For GPT-6 Luna low with automatic feedback, `mid-0006` and `mid-0010` were omitted from the restarted batch after these blocking errors. These two cases are classified as omissions after blocking errors; no direct service error is recorded for them in that condition. All five cases count as unsuccessful evaluations under the planned protocol.